

FINAL PROJECT

Frontend Quizbank

前端題庫刷題系統

純前端 · IndexedDB 本機資料庫 · 單頁應用 (SPA)

第一組 組員：簡嘉佑、石驥惟、李詮彥、邱美綺

線上試用：<https://chuan13.github.io/frontend-quizbank>



報告大綱

AGENDA

01

系統簡介與目標

我們解決什麼問題

02

系統需求規格

五大需求項目

03

功能展示

四大模組 (分組分工)

04

技術架構

SPA · IndexedDB · 模組化

05

系統設計模型

DFD · Use Case · Sequence

06

資安、部署與成果

取捨與總結

系統簡介與目標

OVERVIEW

我們解決的問題

- 市面刷題工具常需註冊、連網或付費
- 自建題庫逐題輸入耗時、難維護
- 錯題複習功能往往不夠彈性

我們的解法

- 純前端、免後端，開啟即用
- JSON 一鍵批次匯入題庫
- 資料存本機 IndexedDB，不外流

多模式

出題策略

未答 / 錯題 / 隨機

量化

學習成效

逐題記錄答對 / 答錯

完整

題庫管理

查看 / 編輯 / 匯入 / 維護

系統需求規格

5 REQUIREMENT AREAS

①

系統名稱

Frontend Quizbank 前端題
庫刷題系統

②

功能性需求

測驗 · 題庫管理 · JSON 匯
入 · 狀態維護

③

UI/UX 設計

RWD · 深色主題 · 卡片式 ·
SPA 切換

④

技術規格

HTML5/CSS3/ES6+ ·
IndexedDB · 原生 JS

⑤

資安要求

輸入驗證 · XSS 跳脫 · 資
料最小化

功能模組總覽

FOUR MODULES · ONE QuizDB



測驗 Quiz

出題設定、抽題、作答、即時回饋與統計



題庫管理 Bank

查看、篩選、排序、編輯、UI 新增題目



JSON 匯入 Importer

檔案上傳或貼上文字，批次匯入題庫



狀態與維護 Maint

資料庫統計、清空錯題、清空全部



測驗 Quiz

FUNCTION DETAIL

主要功能

- 出題範圍：全部題庫 / 指定單一題庫
- 三種出題模式：未答優先、錯題優先、完全隨機
- 單選作答，送出後即時回饋（綠對 / 紅錯）並顯示詳解
- 依選項 order 決定隨機或固定排序，作答後更新統計

主要資料操作

```
QuizDB.getAll()
```

```
QuizDB.updateStats(id, ok)
```

所有模組共用 QuizDB 資料層，操作同一個 IndexedDB。



題庫管理 Bank

FUNCTION DETAIL

主要功能

- 題目列表：顯示題庫、題號、預覽與答對 / 答錯統計
- 可依題庫篩選，依題號或錯題次數排序
- 彈窗編輯題目、選項、正確答案與詳解
- UI 新增單題（可建新題庫、連續新增）、一鍵複製

主要資料操作

```
QuizDB.getAll()
```

```
QuizDB.add(...)
```

```
QuizDB.update(id, patch)
```

```
QuizDB.getBanks()
```

所有模組共用 QuizDB 資料層，操作同一個 IndexedDB。



JSON 匯入 Importer

FUNCTION DETAIL

主要功能

- 兩種來源：檔案上傳 (.json) 或貼上文字
- 匯入前驗證 JSON 格式 (須為陣列)，格式錯誤即攔截
- 可選既有題庫或建立新題庫，自動以檔名帶入
- 批次寫入並自動接續題號，完成後回報匯入題數

主要資料操作

```
QuizDB.addBatch(items, bank)
```

```
QuizDB.getBanks()
```

所有模組共用 QuizDB 資料層，操作同一個 IndexedDB。



狀態與維護 Maint

FUNCTION DETAIL

主要功能

- 資料庫統計：總題數、題庫數、累計答對 / 答錯
- 列出各題庫題數
- 清空所有錯題紀錄 (fail 歸零，題目保留)
- 清空整個資料庫 (二次確認，避免誤刪)

主要資料操作

```
QuizDB.getAll()
```

```
QuizDB.resetAllFails()
```

```
QuizDB.clearAll()
```

所有模組共用 QuizDB 資料層，操作同一個 IndexedDB。

技術架構

ARCHITECTURE



技術要點

- 原生 JavaScript，無前端框架
- HTML5 / CSS3 / ES6+
- IIFE 模組化封裝，避免命名衝突
- IndexedDB 以 Promise / async-await 封裝
- 陣列處理用 map / filter / reduce / find
- 小駝峰命名、中文註解

IndexedDB 資料設計

DATA DESIGN · QuizDB

為什麼選 IndexedDB

- 純前端要保存大量結構化題庫，localStorage 容量小、且只能存字串
- IndexedDB：容量大、可直接存 JavaScript 物件
- 非同步存取，大量讀寫不阻塞畫面
- 支援索引 (index)，篩選與排序更有效率
- 資料留存本機，重整或關閉瀏覽器後不遺失

資料結構與存取層

Object Store：**questions** (keyPath: id 自動遞增)

索引 index：questionBank · questionNumber · correct · fail

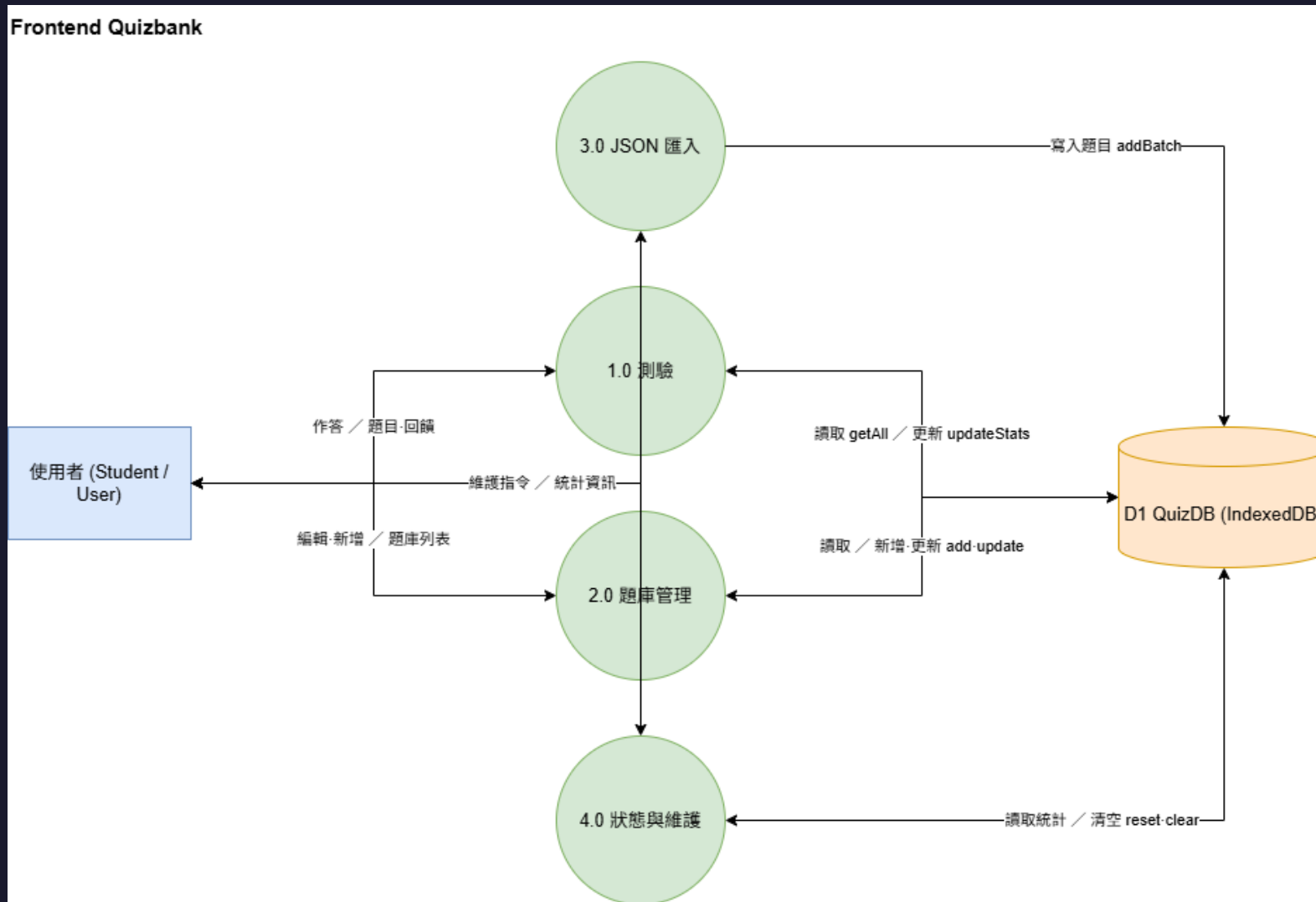
QuizDB 資料層 — 以 Promise 封裝原生 API

```
init() · getAll() · getBanks()  
add() · addBatch()  
update() · updateStats()  
resetAllFails() · clearAll()
```

四個模組共用同一個資料庫，任一模組寫入，其他模組即時可見。

系統設計模型

DIAGRAMS · DRAW.IO



資安設計與取捨

SECURITY

✓ 已落實之防護

- JSON 匯入前驗證格式 (須為陣列) , 錯誤即攔截
- 輸入回填 HTML 屬性時 `escapeAttr()` 跳脫 , 防 XSS
- 破壞性操作採確認對話框 (清空全部二次確認)
- 資料僅存本機、不上傳、不蒐集個資 (資料最小化)

⌚ 因架構而不適用 (主動說明)

- SQL / Command Injection : 無後端與 SQL , 使用 IndexedDB 物件存取 , 無注入面
- 密碼政策 / Cookie HttpOnly : 無帳號登入、不存密碼或 Token , 故不適用

🛡 部署層建議

託管平台啟用 HTTPS。

部署

DEPLOYMENT · GITHUB PAGES

部署方式：GitHub Pages

`chuan13.github.io/frontend-quizbank`

- 純靜態網站，推送即自動發佈
- 僅三個檔案：index.html / app.css / app.js
- 無需建置流程、無伺服器與資料庫
- 免費託管，並提供 HTTPS

部署流程

1

推送程式碼到 GitHub Repository

2

Settings → Pages 選定發佈分支

3

GitHub 自動建置並發佈，取得 HTTPS 網址

純前端三檔，部署近乎零成本；資料存於使用者瀏覽器，伺服器端不留任何資料。

成果總結

RESULTS

4

功能模組

1

單頁 SPA / 3 檔

0

後端伺服器

100%

本機資料

重點回顧

- 完成測驗、題庫管理、JSON 匯入、狀態維護四大功能，共用單一 IndexedDB
- 提供完整需求規格書與設計模型圖

Thank You

Q & A

chuan13.github.io/frontend-quizbank

